

GIỚI THIỆU CHUNG

Câu I.

Phần I.1

1. Mô tả đề bài

Đề bài yêu cầu xây dựng một chương trình thu thập dữ liệu thống kê của tất cả các cầu thủ tham gia thi đấu tại giải bóng đá Ngoại hạng Anh mùa giải 2024 – 2025, với điều kiện chỉ giữ lại những cầu thủ có tổng số phút thi đấu lớn hơn 90 phút. Dữ liệu phải được lấy từ trang web thống kê bóng đá FBref tại địa chỉ <https://fbref.com/en/>.

Toàn bộ kết quả sau khi thu thập và xử lý phải được lưu trữ vào một bảng trong cơ sở dữ liệu SQLite. Cấu trúc bảng được quy định theo nguyên tắc: mỗi cột tương ứng với một chỉ số thống kê của cầu thủ (ví dụ như số trận ra sân, số bàn thắng, số đường chuyền, số phút thi đấu, các chỉ số phòng ngự, tấn công, kiểm soát bóng, v.v.). Đối với bất kỳ chỉ số nào không tồn tại, không được cung cấp cho một cầu thủ cụ thể hoặc không áp dụng (ví dụ thủ môn không có một số chỉ số dành riêng cho cầu thủ tấn công), giá trị trong bảng phải được ghi là chuỗi “N/a”.

2. Phân tích yêu cầu

a) Dữ liệu phạm vi thu thập

Dữ liệu phải được lấy cho toàn bộ cầu thủ thi đấu tại giải Ngoại hạng Anh mùa 2024 – 2025, không chỉ một đội bóng. Điều này đồng nghĩa với việc chương trình phải truy cập trang tổng hợp của giải, sau đó làm việc với các bảng thống kê cấp độ giải đấu (comps/9/2024-2025) thay vì chỉ tập trung vào một câu lạc bộ đơn lẻ.

b) Điều kiện lọc

Chỉ các cầu thủ có tổng số phút thi đấu lớn hơn 90 phút mới được đưa vào kết quả cuối cùng. Do đó, trong dữ liệu thu thập bắt buộc phải có cột liên quan đến số phút thi đấu (ví dụ “Minutes”, “Min”, “Min Played”, tùy theo cách FBref đặt tên), và sau khi gom dữ liệu cho từng cầu thủ, chương trình phải áp dụng bộ lọc này trước khi ghi vào cơ sở dữ liệu.

c) Nguồn dữ liệu và cách trích xuất

FBref tổ chức dữ liệu dưới dạng các bảng HTML trên trang web. Một số bảng có thể được hiển thị trực tiếp, một số khác có thể được đặt trong các đoạn comment HTML nhằm hạn chế việc trích xuất tự động. Vì vậy, chương trình cần có khả năng: tải trang, truy cập đúng phần chứa bảng, đọc mã HTML, phân tích cấu trúc bảng và trích xuất nội dung từng hàng, từng cột.

d) Mô hình lưu trữ

Kết quả cuối cùng phải được lưu vào một bảng SQLite, trong đó:

- Mỗi hàng phải tương ứng với một cầu thủ
- Mỗi cột tương ứng với một chỉ số thống kê
- Giá trị rỗng, thiếu, không áp dụng phải chuẩn hóa thành “N/a” để đảm bảo tính thống nhất, tránh để trống hoặc kiểu dữ liệu không đồng nhất

e) Kỹ thuật cài đặt

Để đáp ứng yêu cầu tự động thu thập dữ liệu từ một trang web động, chương trình cần sử dụng các công cụ như Selenium, Undetected_chromedriver hoặc trình điều khiển trình duyệt tương tự để mô phỏng thao tác của người dùng, kết hợp với thư viện phân tích HTML như BeautifulSoup để xử lý nội dung trang. Khi đã có dữ liệu ở dạng bảng, có thể sử dụng pandas để đưa về dạng DataFrame và sqlite3 để lưu lại vào cơ sở dữ liệu SQLite.

3. Thư viện yêu cầu

- **Undetected_chromedriver:** Khởi tạo Chrome driver có khả năng tránh bị phát hiện là bot.
- **Selenium:** Tự động hóa trình duyệt, tương tác với trang web.
- **BeautifulSoup4:** Parse và phân tích HTML.
- **Pandas:** Xử lý vào thao tác dữ liệu dạng bảng
- **SQLite3:** Lưu trữ dữ liệu vào database
- **Collections:** Tạo dictionary tự động khởi tạo giá trị mặc định

4. Ý tưởng và hướng giải quyết

Ý tưởng tổng quát của bài toán là xây dựng một pipeline tự động gồm các bước: truy cập trang web thống kê, trích xuất dữ liệu từ nhiều bảng khác nhau, hợp nhất dữ liệu theo từng cầu thủ, lọc theo số phút thi đấu rồi lưu kết quả cuối cùng vào cơ sở dữ liệu SQLite với cấu trúc cột tương ứng với từng chỉ số.

Cụ thể, có thể triển khai theo hướng sau:

a) Khởi tạo và cấu hình trình duyệt tự động

Sử dụng **undetected_chromedriver** kết hợp với **Selenium** để khởi tạo một phiên bản trình duyệt Chrome có thể vượt qua một số cơ chế phát hiện bot. Thiết lập thời gian chờ tải trang, phóng to cửa sổ và chèn các khoảng dừng ngẫu nhiên để mô phỏng hành vi người dùng thực tế.

b) Duyệt các trang thống kê theo từng nhóm chỉ số

Từ URL gốc của mùa giải Ngoại hạng Anh 2024 – 2025, kết hợp với các đường dẫn con tương ứng với từng loại bảng thống kê (standard, shooting, passing, passing types, gca, defense, possession, playing_time, misc, goalkeeping, goalkeeping_adv). Đối với mỗi loại bảng, chương trình tải trang, chờ phần tử chứa bảng xuất hiện, sau đó trích xuất nội dung HTML của container tương ứng.

c) Phân tích cấu trúc bảng và trích xuất dữ liệu

Sử dụng **BeautifulSoup** để phân tích **HTML**, xác định hàng tiêu đề (**thead**) và nội dung (**tbody**). Tên cột được lấy từ thuộc tính data-stat hoặc từ nội dung của thẻ **th**. Mỗi hàng dữ liệu được chuyển thành một dict, trong đó các trường quan trọng như tên cầu thủ, câu lạc bộ, quốc tịch, tuổi, vị trí được chuẩn hóa lại tên. Với các bảng khác nhau, có thể thêm tiền tố vào tên chỉ số để phân biệt nguồn gốc (ví dụ: shooting__, passing__, defense__), đảm bảo không trùng tên cột khi hợp nhất.

d) Hợp nhất dữ liệu theo từng cầu thủ

Sử dụng một cấu trúc dữ liệu dạng **defaultdict(dict)** với khóa là cặp (tên cầu thủ, câu lạc bộ). Mỗi lần đọc xong một bảng thống kê, chương trình lần lượt hợp nhất các chỉ số mới vào bản ghi tương ứng của cầu thủ. Các thông tin cơ bản lấy từ bảng standard được xem là chuẩn và chỉ bị ghi đè khi còn thiếu.

e) Lọc theo số phút thi đấu

Sau khi có đầy đủ dữ liệu, chương trình kiểm tra các trường liên quan đến số phút thi đấu, chuyển về số nguyên và chỉ giữ lại những cầu thủ có tổng số phút thi đấu lớn hơn 90. Đây là bước ràng lọc quan trọng nhằm loại bỏ những trường hợp ít ra sân, giúp dữ liệu đầu ra ổn định, có ý nghĩa hơn cho phân tích thống kê.

f) Chuẩn hóa và lưu dữ liệu vào SQLite

Từ cấu trúc dict của tất cả cầu thủ, chương trình chuyển sang **DataFrame** của pandas, xử lý các cột trùng lặp và đổi tên các cột cho thống nhất (ví dụ đổi squad thành club). Toàn bộ giá trị rỗng, **NaN** hoặc không hợp lệ được chuyển thành chuỗi "N/a" đúng yêu cầu đề bài. Cuối cùng, chương trình kết nối tới cơ sở dữ liệu SQLite, tạo mới bảng kết quả với mỗi cột tương ứng một chỉ số và chèn lần lượt từng hàng dữ liệu vào.

5. Cấu Trúc Chương Trình

Chương trình được tổ chức thành các hàm riêng biệt, mỗi hàm đảm nhận một giai đoạn trong quy trình xử lý dữ liệu từ FBref.

a) Nhóm hàm cấu hình và anti-detection

build_driver()

- Khởi tạo Chrome driver với undetected_chromedriver để tránh bị phát hiện là bot.
- Quy trình chính:
- Thử khởi tạo với Chrome version 141, nếu thất bại thì auto-detect version hiện tại
- Set timeout 90 giây cho page load và script execution
- Maximize cửa sổ browser để đảm bảo tất cả elements được render đầy đủ
- Kết quả: Trả về Chrome driver instance sẵn sàng sử dụng

human_pause(min_delay, max_delay)

- Giúp trình duyệt mô phỏng hành vi người dùng thực bằng cách tạo khoảng dừng ngẫu nhiên.
- Quy trình:

- Tạo delay ngẫu nhiên từ 0.7 đến 1.6 giây (mặc định)
- Sử dụng `random.uniform()` để tránh pattern đều đặn
- Mục đích: Tránh bị FBref phát hiện là bot và rate limit/IP ban

b) Hàm tải và thu thập HTML

`extract_table_html(driver, container_id, table_id)`

- Là hàm trung tâm của giai đoạn Extract, trích xuất HTML của bảng thống kê từ trang web.
- Quy trình chính:
 - Tìm container element theo `container_id` bằng Selenium
 - Lấy innerHTML của container
 - Parse HTML bằng BeautifulSoup với parser lxml
 - Tìm bảng theo `table_id` - trước tiên tìm trực tiếp trong DOM
 - Nếu không tìm thấy, tìm trong HTML comments `<!-- -->` (FBref ẩn một số bảng trong comments)
 - Trả về HTML string của bảng hoàn chỉnh
 - Kết quả: HTML string của bảng hoặc None nếu không tìm thấy

c) Hàm trích xuất và phân tích dữ liệu bảng

`parse_table_html(table_html, page_type)`

- Sử dụng BeautifulSoup để phân tích mã HTML và chuyển đổi thành dữ liệu có cấu trúc.
- Quy trình:
 - Validate input: kiểm tra `table_html` có hợp lệ
 - Trích xuất headers từ dòng cuối cùng của `<thead>` (FBref sử dụng multi-level headers)
 - Ưu tiên lấy giá trị từ attribute `data-stat` thay vì text hiển thị
 - Duyệt qua từng row trong `<tbody>`, bỏ qua các header rows được chèn giữa dữ liệu
 - Xử lý đặc biệt cho cột "player" (lấy cả tên và link) và "squad/team" (normalize thành "squad")
 - Gọi `_process_stat_cell()` để xử lý các cells còn lại

- Kết quả: List of dictionaries, mỗi dictionary đại diện cho một cầu thủ với đầy đủ thông kê

`_process_stat_cell(row_data, key, value, page_type)`

- Chuẩn hóa tên cột và giá trị cho từng cell dữ liệu.
- Quy trình:
- Định nghĩa mapping cho các cột cơ bản: born → birth_year, pos → position, nationality → nation
- Chỉ normalize khi crawl từ bảng "standard" để tránh tạo cột trùng lặp (ví dụ: shooting__age, passing__age)
- Các cột khác giữ nguyên tên gốc
- Kết quả: row_data được cập nhật với tên cột đã chuẩn hóa

d) Nhóm hàm xử lý và làm sạch dữ liệu

`merge_player_stats(master_data, new_rows, page_type)`

- Hợp nhất dữ liệu từ 11 bảng thông kê khác nhau thành một dataset thống nhất.
- Quy trình:
- Sử dụng tuple (player, squad) làm unique key để phân biệt cầu thủ chuyển đội giữa mùa
- Sử dụng defaultdict(dict) để tự động tạo entry mới khi cần
- Merge stats từ các page khác nhau với naming convention:
- Bảng "standard": goals, assists, minutes
- Bảng khác: shooting__shots, passing__passes_completed, defense__tackles
- Replace empty values thành "N/a" để phân biệt với giá trị 0
- Kết quả: master_data chứa tất cả stats của tất cả cầu thủ, mỗi cầu thủ là một dictionary

`filter_by_minutes(player_data, threshold)`

- Lọc dữ liệu chỉ giữ các cầu thủ có số phút thi đấu lớn hơn ngưỡng (mặc định 90 phút).

- Quy trình:
- Tìm cột minutes trong 3 cột khả dĩ với thứ tự ưu tiên:
- minutes (từ bảng standard - cầu thủ thường)
- playing_time__minutes (từ bảng playing_time - chi tiết hơn)
- goalkeeping__gk_minutes (từ bảng goalkeeper - chỉ có cho thủ môn)
- Convert string thành integer, loại bỏ dấu phẩy nếu có ("1,234" → 1234)
- Lọc chỉ giữ cầu thủ có minutes > threshold
- Kết quả: Dictionary đã được lọc, đảm bảo dữ liệu đại diện cho các cầu thủ ra sân thường xuyên

clean_duplicate_columns(dataframe)

- Làm sạch DataFrame bằng cách loại bỏ các cột có dữ liệu trùng lặp.
- Quy trình:
- Định nghĩa các nhóm cột có khả năng trùng lặp: games/matches, age, position, nation...
- Tìm tất cả cột trong DataFrame khớp với mỗi nhóm
- Giữ lại cột ưu tiên (primary_key), xóa các cột còn lại
- Exclude các cột có pattern đặc biệt: "90", "per_90", "starts", "subs" (không phải duplicate)
- Kết quả: DataFrame đã loại bỏ duplicate columns, giảm từ ~270 cột xuống ~180 cột

e) Hàm lưu trữ dữ liệu

`save_to_sqlite(dataframe, db_path, table_name)`

- Lưu DataFrame đã xử lý vào SQLite database với cấu trúc tối ưu.
- Quy trình:
- Gọi clean_duplicate_columns() để làm sạch lần cuối
- Rename cột "squad" → "club" (thuật ngữ phổ biến hơn)
- Sắp xếp cột theo thứ tự logic:
- Base columns ở đầu: player, club, age, birth_year, nation, position
- Stat columns theo category: standard, shooting, passing, defense...
- DROP TABLE cũ nếu tồn tại
- CREATE TABLE mới với tất cả cột type TEXT

- Insert data bằng executemany() (nhanh hơn nhiều lần so với loop)
- Replace NULL/NaN/None thành "N/a" để đồng nhất dữ liệu
- Kết quả: File SQLite với bảng player_stats_cleaned chứa dữ liệu đã chuẩn hóa

f) Hàm main()

- Là trái tim của chương trình, điều phối toàn bộ quy trình ETL (Extract, Transform, Load).
- Quy trình chính:
- Khởi tạo Chrome driver với build_driver()
- Khởi tạo defaultdict(dict) để lưu trữ dữ liệu tất cả cầu thủ
- Duyệt qua 11 trang thống kê trong STAT_PAGES:
- Gọi driver.get(url) để navigate đến trang
- Wait cho container element load bằng WebDriverWait
- Gọi extract_table_html() để lấy HTML bảng
- Gọi parse_table_html() để parse thành structured data
- Gọi merge_player_stats() để gộp vào master dataset
- Random pause để tránh rate limit
- Gọi filter_by_minutes() để lọc cầu thủ có > 90 phút
- Convert sang pandas DataFrame
- Drop cột player_link (không cần thiết cho analysis)
- Gọi save_to_sqlite() để lưu vào database
- Finally block: đóng driver dù có lỗi hay không để clean up resources
- Kết quả: File SQLite fbref_pl_2024_2025_full.sqlite chứa ~500 cầu thủ với ~180 cột thống kê

6. Khó khăn và giải pháp

1. FBref Anti-Bot Protection

Khó khăn

- Cloudflare Protection: FBref sử dụng hệ thống Cloudflare để chống bot tự động
- Selenium thường bị phát hiện: Trình duyệt thông thường qua Selenium dễ bị detect và block

- Rate Limiting: Các request gửi quá nhanh sẽ bị server từ chối

Hướng giải quyết

1. Sử dụng undetected_chromedriver thay vì Selenium thường:

- Thư viện undetected_chromedriver được thiết kế đặc biệt để tránh bị phát hiện
- Loại bỏ các dấu hiệu nhận dạng bot như webdriver flags, automation extensions
- Giả lập hành vi trình duyệt thật một cách tự nhiên hơn

2. Áp dụng random delays giữa các requests:

- Tạo khoảng dừng ngẫu nhiên từ 0.7 đến 1.6 giây giữa mỗi request
- Mô phỏng hành vi người dùng thật (không ai click quá nhanh đều đặn)
- Tránh pattern recognition của hệ thống anti-bot

3. Không sử dụng headless mode:

- Chạy browser với giao diện đầy đủ (không ẩn)
- Headless mode thường là dấu hiệu rõ ràng của bot
- Browser hiển thị giúp tăng độ tin cậy

4. Set timeout hợp lý:

- Cho phép trang web đủ thời gian load và render (90 giây)
- Tránh timeout sớm gây lỗi không cần thiết

2. Dữ Liệu Ẩn Trong HTML Comments

Khó khăn

- Bảng không hiển thị trực tiếp: Một số bảng thống kê được FBref ẩn trong HTML comments <!-- -->
- Chỉ load khi user click: Bảng chỉ được uncomment khi người dùng click vào tab tương ứng
- Selenium không thấy: Nếu chỉ parse DOM thông thường sẽ bị thiếu dữ liệu (30-40%)

Hướng giải quyết

1. Lấy toàn bộ innerHTML của container:

- Sử dụng Selenium để lấy innerHTML thay vì chỉ lấy các elements hiển thị
- innerHTML chứa cả phần comment nên không bỏ sót dữ liệu

2. Parse cả HTML comments:

- Sử dụng BeautifulSoup để tìm tất cả HTML comments trong source code
- Comments có dạng `<!-- <table>...</table> -->`
- FBref ẩn bảng trong comments để giảm tải trang ban đầu

3. Tìm kiếm 2 lần (fallback mechanism):

- Bước 1: Tìm bảng trực tiếp trong DOM (các bảng đã được render)
- Bước 2: Nếu không tìm thấy, tìm trong comments (các bảng ẩn)
- Đảm bảo thu thập được tất cả bảng

4. Parse lại HTML từ comment:

- Khi tìm thấy table trong comment, parse lại content của comment
- Chuyển string HTML thành BeautifulSoup object để xử lý tiếp

3. Cột Trùng Lặp Từ Nhiều Bảng

Khó khăn

- 11 bảng, mỗi bảng có cột lặp: Mỗi bảng thống kê đều có các cột: age, position, team, matches...
- Dữ liệu trùng 100%: Nếu không xử lý sẽ có 11 cột age, 11 cột position với data giống hệt nhau
- DataFrame phình to: ~270 cột thay vì ~180 cột cần thiết
- Tốn memory và khó phân tích

Hướng giải quyết

1. Normalize cột cơ bản ngay khi crawl:

- Xác định các cột cơ bản: age, position, team, nation, birth_year...
- Tạo mapping chuẩn cho các tên cột biến thể (ví dụ: "born" → "birth_year", "pos" → "position")

- Chỉ lấy giá trị từ bảng "standard" cho các cột cơ bản
- Các bảng khác sẽ không tạo cột trùng lặp này

2. Thêm prefix cho cột của mỗi bảng:

- Bảng "standard": giữ nguyên tên cột (goals, assists, minutes...)
- Các bảng khác: thêm prefix theo loại (shooting__shots, passing__passes, defense__tackles...)
- Để phân biệt cột nào từ bảng nào

3. Định nghĩa nhóm cột duplicate:

- Tạo dictionary các nhóm cột có khả năng trùng lặp
- Mỗi nhóm có 1 cột ưu tiên (primary key)
- Ví dụ: nhóm "age" gồm [age, shooting__age, passing__age...]

4. Phát hiện và xóa cột trùng:

- Tìm tất cả cột trong DataFrame khớp với từng nhóm
- Giữ lại cột ưu tiên (primary key)
- Xóa tất cả các cột còn lại trong nhóm
- Exclude các cột có pattern đặc biệt (90, per_90, starts, subs)

5. Thực hiện 2 lần:

- Lần 1: Khi merge dữ liệu từ 11 bảng
- Lần 2: Trước khi save vào SQLite (clean lần cuối)

4. Multi-Level Table Headers

Khó khăn

- Header có nhiều cấp: FBref sử dụng header 2-3 cấp (Performance > Goals, Expected > xG...)
- Không biết lấy dòng nào: Khó xác định dòng nào chứa tên cột thật
- Colspan gây nhầm lẫn: Các cell với colspan làm lệch số lượng cột
- Text hiển thị khác data thật: Text hiển thị là "Gls" nhưng tên cột thực tế là "goals"

Hướng giải quyết

1. Luôn lấy dòng cuối cùng của thead:

- Thead có nhiều rows, nhưng chỉ row cuối chứa tên cột chi tiết
- Các row trên chỉ là grouping/category
- Parse tất cả rows và chọn row cuối (index = -1)

2. Ưu tiên attribute data-stat:

- FBref lưu tên cột thật trong attribute data-stat của thẻ <th>
- Text hiển thị cho người đọc (viết tắt, có định dạng)
- Attribute cho máy đọc (tên đầy đủ, không format)
- Luôn lấy data-stat trước, nếu không có mới lấy text

3. Fallback mechanism:

- Nếu không có data-stat attribute
- Lấy text content của <th> và normalize
- Đảm bảo luôn có tên cột dù format thế nào

4. Xử lý colspan:

- Không cần xử lý đặc biệt vì chỉ lấy row cuối
- Row cuối không có colspan, mỗi cell = 1 cột
- Các row trên có colspan nhưng không dung

Phần I.2 - Thu Thập Giá Trị Chuyển Nhượng Cầu Thủ

1. Mô tả đề bài

Sau khi hoàn thành phần I.1 với dữ liệu thống kê chi tiết về các cầu thủ Premier League mùa 2024-2025, phần I.2 yêu cầu bổ sung thêm thông tin về giá trị chuyển nhượng (market value) của từng cầu thủ vào cơ sở dữ liệu.

Cụ thể, chương trình cần:

- Lấy danh sách tất cả cầu thủ từ bảng `player_stats_cleaned` đã tạo ở phần I.1
- Thu thập giá trị chuyển nhượng hiện tại của từng cầu thủ từ website `footballtransfers.com`

- Lưu kết quả vào một bảng mới `player_transfer_values_2024_2025` trong cùng database SQLite
- Đảm bảo xử lý các trường hợp: tên cầu thủ có dấu, bí danh, không tìm thấy thông tin

Kết quả cuối cùng phải bao gồm các trường: tên cầu thủ, giá trị chuyển nhượng (market value), cầu lạc bộ hiện tại, và ngày cập nhật thông tin.

2. Phân tích yêu cầu

a) Nguồn dữ liệu đầu vào

Dữ liệu đầu vào là danh sách cầu thủ từ bảng `player_stats_cleaned` trong file SQLite `fbref_pl_2024_2025_full.sqlite`. Chương trình cần query để lấy danh sách tên cầu thủ duy nhất và lặp qua từng cầu thủ để thu thập thông tin giá trị chuyển nhượng.

b) Nguồn dữ liệu thu thập

Website [footballtransfers.com](https://www.footballtransfers.com) cung cấp thông tin giá trị chuyển nhượng của cầu thủ. Cấu trúc URL có dạng:

- URL trực tiếp: `https://www.footballtransfers.com/en/players/{player-slug}`
- URL tìm kiếm: `https://www.footballtransfers.com/en/players?q={player-name}`

Trong đó `player-slug` là tên cầu thủ được chuẩn hóa

c) Xử lý tên cầu thủ

Tên cầu thủ trong database có thể:

- Có dấu (ví dụ: "Emiliano Martínez", "João Pedro")
- Có nhiều phần (ví dụ: "Mohamed Salah")
- Có alias khác nhau trên website (ví dụ: "Bruno Fernandes" có thể là "bruno-fernandes-1", "bruno-fernandes-2")

Do đó, chương trình cần:

- Chuẩn hóa tên thành slug (bỏ dấu, lowercase, thay space bằng -)
- Thử nhiều variants (thêm -1, -2, -3 vào cuối slug)

- Sử dụng search API nếu không tìm thấy trực tiếp
- Log các alias để tái sử dụng

d) Thông tin cần thu thập

Từ trang profile của cầu thủ, cần extract:

- Market Value: Giá trị chuyển nhượng (định dạng: €10.0m, €50.0k...)
- Club: Câu lạc bộ hiện tại
- Last Update: Ngày cập nhật thông tin gần nhất

e) Xử lý trường hợp đặc biệt

- Không tìm thấy cầu thủ: Ghi giá trị "N/a" cho tất cả các trường
- Redirect: Website có thể redirect từ URL này sang URL khác, cần follow redirect và log lại
- Timeout: Request quá lâu, cần retry với timeout cao hơn
- Cầu thủ trùng tên: Sử dụng search results để chọn đúng cầu thủ

f) Mô hình lưu trữ

Tạo bảng mới `player_transfer_values_2024_2025` với cấu trúc:

Cột	Kiểu dữ liệu	Mô tả
player	TEXT	Tên cầu thủ
market_value	TEXT	Giá trị chuyển nhượng (€)
club	TEXT	Câu lạc bộ hiện tại
last_update	TEXT	Ngày cập nhật

Tất cả cột đều dùng TEXT để đơn giản hóa, giá trị thiếu ghi "N/a".

3. Thư viện yêu cầu

- sqlite3: Kết nối và thao tác với SQLite database
- requests: Gửi HTTP requests để tải trang web
- BeautifulSoup4: Parse và phân tích HTML
- unidecode: Chuyển đổi ký tự Unicode sang ASCII
- re: Xử lý regular expressions để chuẩn hóa tên

- urllib.parse: Encode URL parameters cho search query

4. Ý tưởng và hướng giải quyết

Tổng quan

Xây dựng một pipeline tự động gồm các bước: lấy danh sách cầu thủ từ database, chuẩn hóa tên, thử nhiều URL variants để tìm thông tin, parse HTML để extract market value, và lưu kết quả vào database.

Chi tiết triển khai

a) Khởi tạo database và bảng kết quả

- Kết nối tới database SQLite hiện có
- Tạo bảng player_transfer_values_2024_2025 nếu chưa tồn tại
- Xóa dữ liệu cũ để đảm bảo chạy fresh mỗi lần

b) Chuẩn hóa tên cầu thủ thành slug

Sử dụng unidecode để:

- Chuyển Unicode → ASCII (Martínez → Martinez)
- Lowercase tất cả ký tự
- Loại bỏ ký tự đặc biệt (giữ chữ cái, số, space, dấu gạch ngang)
- Thay space bằng dấu gạch ngang
- Kết quả: "Mohamed Salah" → "mohamed-salah"

c) Sử dụng search API khi không tìm thấy trực tiếp

Nếu tất cả variants đều thất bại:

- Gửi request tới `https://www.footballtransfers.com/en/players?q={player-name}`
- Parse HTML để tìm link đầu tiên trong search results
- Extract href và crawl trang đó
- Hoặc lấy luôn thông tin từ search results (có market value preview)

d) Parse HTML để extract thông tin

Sử dụng BeautifulSoup với CSS selectors:

- Market Value: `.player-value` `.player-tag` hoặc `.playerInfo-value`

- Club: `.playerInfoTop-bar__club` hoặc `.playerInfoTop-bar__team`
- Last Update: Tìm text chứa "Last update" và extract phần sau dấu :

Validate: Market value phải bắt đầu bằng € để đảm bảo trang đúng.

e) Xử lý timeout và retry

- Default timeout: 15 giây
- Nếu timeout, retry 1 lần với timeout 20 giây
- Nếu vẫn fail, ghi "N/a" và tiếp tục cầu thủ khác
- Không dừng chương trình vì một cầu thủ

h) Lưu từng record vào database

Sau mỗi lần crawl thành công một cầu thủ:

- Insert ngay vào database (không đợi hết)
- Commit transaction để đảm bảo dữ liệu được lưu
- Tránh mất dữ liệu nếu chương trình bị gián đoạn

5. Cấu trúc chương trình

Chương trình được tổ chức thành các hàm chức năng riêng biệt:

a) Nhóm hàm database

`init_db()`

- Tạo bảng `player_transfer_values_2024_2025` nếu chưa tồn tại
- Định nghĩa schema với 4 cột: `player`, `market_value`, `club`, `last_update`
- Tất cả cột type TEXT

`clear_table()`

- Xóa toàn bộ dữ liệu cũ trong bảng
- Thực hiện trước khi bắt đầu crawl mới
- Đảm bảo không có dữ liệu duplicate

`insert_record(player, value, club, date)`

- Insert một record vào database
- Commit ngay sau khi insert

- Không batch để tránh mất data nếu crash

b) Nhóm hàm xử lý tên và URL

make_slug(name)

- Chuyển tên Unicode sang ASCII bằng unidecode()
- Lowercase toàn bộ
- Loại bỏ ký tự đặc biệt (chỉ giữ a-z, 0-9, space, dash)
- Replace space bằng -
- Kết quả: slug chuẩn để tạo URL

log_alias(original, alias_url)

- Ghi lại mapping giữa tên gốc và URL thực tế
- Append vào file alias_detected.txt
- Hữu ích cho debug và tối ưu hóa

c) Nhóm hàm crawl dữ liệu

crawl_page(url)

- Gửi GET request với User-Agent header
- Follow redirects (allow_redirects=True)
- Nếu URL redirect, log alias
- Parse HTML bằng BeautifulSoup
- Extract market value, club, last_update bằng CSS selectors
- Validate: market value phải bắt đầu bằng €
- Return tuple (value, club, date) hoặc None nếu fail
- Xử lý timeout: retry 1 lần với timeout cao hơn

crawl_from_search_results(soup)

- Parse search results page
- Tìm item đầu tiên trong listing
- Extract market value và club trực tiếp từ search preview
- Return tuple (value, club, "Approx") nếu tìm thấy
- Return None nếu không tìm thấy

fetch_player_value(player)

- Hàm chính để lấy thông tin của 1 cầu thủ
- Quy trình:
 1. Tạo slug từ tên cầu thủ
 2. Thử 4 URL variants: base, -1, -2, -3
 3. Nếu tìm thấy, return ngay
 4. Nếu không, search qua search API
 5. Parse link đầu tiên trong search results
 6. Crawl trang đó hoặc lấy từ search preview
 7. Nếu vẫn không có, return ("N/a", "N/a", "N/a")

d) Hàm main()

main()

- Khởi tạo database và clear bảng cũ
- Query danh sách cầu thủ **DISTINCT** từ player_stats_cleaned
- In tổng số cầu thủ cần crawl
- Khởi tạo set seen để tránh crawl duplicate
- Loop qua từng cầu thủ:
 - Check xem đã crawl chưa (trong seen)
 - In progress: [1/500] Mohamed Salah
 - Gọi **fetch_player_value()**
 - Insert vào database
 - Add vào seen
 - Catch exception và ghi "N/a" nếu có lỗi
 - In thông báo hoàn tất

6. Khó khăn và giải pháp

1. Tên Cầu Thủ Có Dấu và Biến Thể

Khó khăn:

- Tên cầu thủ trong database có ký tự Unicode (dấu): "João Félix", "Emiliano Martínez"

- URL slug không dùng Unicode, cần chuyển sang ASCII
- Nhiều cách chuyển đổi khác nhau có thể dẫn đến URL sai

Hướng giải quyết:

1. Sử dụng thư viện unidecode:

- Thư viện được thiết kế chuyên biệt cho việc chuyển Unicode → ASCII
- Xử lý đúng hầu hết các ký tự đặc biệt của các ngôn ngữ
- Ví dụ: "José" → "Jose", "Müller" → "Muller"

2. Chuẩn hóa thống nhất:

- Lowercase toàn bộ ký tự
- Loại bỏ tất cả ký tự không phải chữ cái, số, space, dash
- Replace space bằng - để tạo URL-friendly slug

3. Xử lý edge cases:

- Xóa space thừa ở đầu/cuối
- Xóa nhiều dash liên tiếp thành 1 dash
- Đảm bảo slug không bắt đầu/kết thúc bằng dash

2. Cầu Thủ Trùng Tên (Duplicate Names)

Khó khăn:

- Nhiều cầu thủ có cùng tên: "Gabriel", "Pedro", "Bruno Fernandes"
- Website thêm suffix -1, -2, -3 để phân biệt
- Không biết trước suffix nào đúng cho cầu thủ cụ thể

Hướng giải quyết:

1. Thử tuần tự nhiều variants:

- Tạo list URL với các suffix: base, -1, -2, -3
- Loop qua từng URL theo thứ tự
- Dừng ngay khi tìm thấy trang hợp lệ (có market value)

2. Validate kết quả:

- Kiểm tra market value phải bắt đầu bằng €

- Nếu không có €, coi như trang không hợp lệ
- Continue thử URL tiếp theo

3. Early stopping:

- Không cần thử tất cả 4 URLs nếu đã tìm thấy
- Return ngay khi có kết quả hợp lệ
- Tiết kiệm thời gian và requests

4. Log alias để tái sử dụng:

- Ghi lại mapping: tên gốc → URL thực tế
- Có thể build dictionary để optimize lần chạy sau
- Giúp debug khi có vấn đề

Kết quả đạt được:

- Tìm đúng hầu hết cầu thủ dù trùng tên
- Không bỏ sót do chọn sai variant
- Có thể optimize dần theo thời gian

3. Request Timeout và Network Errors

Khó khăn:

- Request có thể timeout (server chậm, mạng kém)
- Connection errors ngẫu nhiên
- Server có thể trả về HTTP error codes (500, 503...)

Hướng giải quyết:

1. Set timeout hợp lý:

- Default timeout: 15 giây (đủ cho hầu hết cases)
- Không quá thấp (dễ timeout)
- Không quá cao (chờ lâu khi thực sự có vấn đề)

2. Exception handling:

- Wrap mọi network call trong try-except
- Catch Exception rộng để cover tất cả cases

- Log error nhưng không dừng program

Kết quả đạt được:

- Program chạy được dù network không ổn định
- Tự động retry khi có thể
- Không mất toàn bộ data vì 1 request fail

Câu II.

Tổng quan

Part 2 cung cấp các công cụ để truy vấn và phân tích dữ liệu Premier League đã thu thập ở Part 1:

- Part 2_1: RESTful API (FastAPI) để truy vấn database
- Part 2_2: CLI tool để tra cứu và export CSV

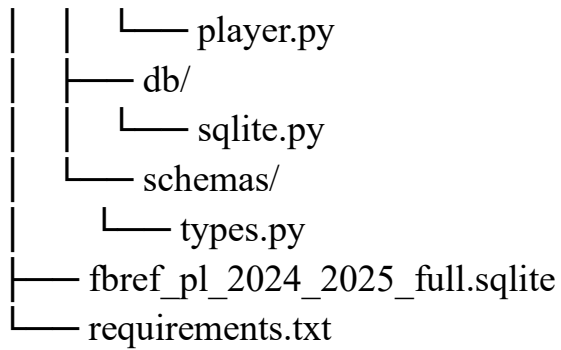
Mục tiêu

- Xây dựng REST API với FastAPI để expose dữ liệu
- Hỗ trợ tìm kiếm theo tên cầu thủ và câu lạc bộ
- Asynchronous database operations với aiosqlite
- CLI tool để query API và export CSV
- Auto-generate API documentation (Swagger/ReDoc)

1. Phần 2_1: FastAPI REST API

Cấu trúc thư mục

```
Part 2_1/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── core/
│   │   └── config.py
│   ├── routers/
│   │   └── players.py
│   └── crud/
```



Công nghệ sử dụng

fastapi>=0.104.0

uvicorn[standard]>=0.24.0

aiosqlite>=0.19.0

API Endpoints

1. GET /players

Tìm kiếm cầu thủ theo tên (LIKE search)

Request:

GET /players?name=haaland

Response:

```
{
  "player": "Erling Haaland",
  "squad": "Manchester City",
  "age": "24",
  "birth_year": "2000",
  "nation": "NOR",
  "position": "FW",
  "standard__games": "31",
  "standard__goals": "27",
  "standard__assists": "5",
  "standard__minutes": "2593",
  ...
}
```

Parameters:

- name: Tên cầu thủ (case-insensitive, partial match)

Features:

- Trả về tất cả kết quả phù hợp
- Case-insensitive search
- Partial matching: "salah" → "Mohamed Salah"
- Trả về đầy đủ 231 cột

2. GET /players/by-club/{club}

Lấy toàn bộ cầu thủ của một câu lạc bộ

Request:

GET /players/by-club/Arsenal?exact=true

Parameters:

- Club (path, required): Tên câu lạc bộ
- Exact (query, default=true): true = Exact match, false = LIKE search

Features:

- Trả về toàn đội hình (thường 20-30 cầu thủ)
- Exact match mặc định
- Option cho fuzzy search

Chi tiết kỹ thuật

Quản lý vòng đời

```
1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     async with aisqlite.connect(str(DB_FILE)) as conn:
4         conn.row_factory = aisqlite.Row
5         await ensure_source_table(conn)
6         await create_indexes(conn)
7         print(f"Kết nối database đã sẵn sàng: {DB_FILE}")
8
9     yield
10
11     await close_db()
12     print("Đã đóng kết nối database")
```

Lợi ích:

- Kiểm tra database availability khi startup
- Tạo indexes tự động (player, squad)
- Graceful shutdown

Hoạt động Cơ sở dữ liệu Bất đồng bộ

```
1 async def get_db() -> AsyncGenerator[aisqlite.Connection, None]:
2     async with aisqlite.connect(str(DB_FILE)) as conn:
3         conn.row_factory = aisqlite.Row
4         yield conn
```

Lợi ích:

- Connection per request (thread-safe)
- Tự động close connection
- Row factory cho dict output

Tiêm phụ thuộc

```
1 @router.get("", summary="Tìm cầu thủ theo tên")
2 async def by_name(
3     name: str = Query(..., description="Tên cầu thủ, ví dụ: 'haaland'"),
4     db: aioredis.Connection = Depends(get_db),
5 ) -> List[Dict[str, Any]]:
6     """Tìm kiếm cầu thủ theo tên (trả về tất cả kết quả phù hợp)"""
7     rows = await fetch_by_name(db, name=name)
8     if not rows:
9         raise HTTPException(status_code=404, detail=f"Không tìm thấy cầu thủ phù hợp với tên '{name}'.")
10    return rows
```

Lợi ích:

- Chia nhỏ thành phần
- Dễ dàng kiểm thử
- Làm sạch code

Tài liệu được sinh tự động

FastAPI tự động tạo interactive docs:

- Swagger UI: <http://127.0.0.1:8000/docs> - Interactive API testing
- ReDoc: <http://127.0.0.1:8000/redoc> - Clean, readable

2. Phần 2_2: Nslookup

Chức năng

Script lookup.py cung cấp command-line interface để:

- Query FastAPI từ terminal
- Hiển thị kết quả dạng table
- Export kết quả ra CSV file

Sử dụng

Tìm kiếm theo tên cầu thủ

```
python lookup.py --name "Bukayo Saka"
```

Output:

```
[
  {
```

```
"player": "Bukayo Saka",
"club": "Arsenal",
"age": "22",
"birth_year": "2001",
"nation": "engENG",
....
}
]
```

CSV đã lưu: bukayo_saka.csv

Tìm kiếm theo câu lạc bộ

```
python lookup.py --club "Arsenal"
```

Output:

```
[
{
  "player": "Riccardo Calafiori",
  "club": "Arsenal",
  "age": "22",
  "birth_year": "2002",
  "nation": "itITA",
  "position":
  "DF",...
},
{
  "player": "Kai Havertz",
```

```
"club": "Arsenal",
"age": "25",
"birth_year": "1999",
"nation": "deGER",
"position": "FW,MF",
"shooting__goals":
"9",
}.....
}
]
```

CSV đã lưu: arsenal.csv

Tính năng

1. Tự động đổi tên

- Input: --name "Mohamed Salah"
- Output: mohamed_salah.csv
- Slugify: normalize Unicode, replace spaces với underscore

2. Xử lý lỗi

- Connection error: Gợi ý chạy server
- API error: Hiển thị status code và detail
- Missing data: Thông báo rõ ràng

3. Hỗ trợ Unicode

with open(filename, "w", newline="", encoding="utf-8-sig") as f:

- UTF-8 with BOM cho Excel compatibility
- Hỗ trợ tên có dấu, ký tự đặc biệt

3. So sánh FastAPI và Flask

1. So sánh tổng quan

Tiêu chí	FastAPI	Flask
Hiệu suất	Rất cao (xử lý đồng thời)	Thấp hơn (xử lý tuần tự)
Tài liệu tự động	Tự động (Swagger/ReDoc)	Phải viết thủ công
Kiểm tra kiểu dữ liệu	Có (báo lỗi tự động)	Không
Kiểm tra giá trị	Tự động	Phải code thủ công
Xử lý đồng thời	Có sẵn async/await	Cần thư viện thêm

2. Chi tiết các ưu điểm của FastAPI

2.1. Xử lý đồng thời (Async/Await) - Hiệu suất vượt trội

FastAPI

```
1 async def fetch_by_name(conn: aiosqlite.Connection, name: str) -> List[Dict]:
2     sql = f"SELECT * FROM [{SOURCE_TABLE}] WHERE LOWER(player) LIKE LOWER(?)"
3     async with conn.execute(sql, (f"%{name}%",)) as cursor:
4         rows = await cursor.fetchall()
5         return [dict(row) for row in rows]
```

```
1 @router.get("", summary="Tìm cầu thủ theo tên")
2 async def by_name(
3     name: str = Query(..., description="Tên cầu thủ, ví dụ: 'haaland'"),
4     db: aiosqlite.Connection = Depends(get_db),
5 ) -> List[Dict[str, Any]]:
6     """Tìm kiếm cầu thủ theo tên (trả về tất cả kết quả phù hợp)"""
7     rows = await fetch_by_name(db, name=name)
8     if not rows:
9         raise HTTPException(status_code=404, detail=f"Không tìm thấy cầu thủ phù hợp với tên '{name}'.")
10    return rows
```

Lợi ích:

- FastAPI: Xử lý nhiều yêu cầu cùng lúc, không bị chặn

- Flask: Mỗi yêu cầu phải đợi nhau, bị chặn
- Hiệu suất: FastAPI nhanh hơn 2-3 lần khi có nhiều người dùng cùng lúc

2.2. Tài liệu tự động

FastAPI:

- Swagger UI: <http://127.0.0.1:8000/docs> - Giao diện để test API
- ReDoc: <http://127.0.0.1:8000/redoc> - Tài liệu đẹp để đọc
- OpenAPI schema tự động sinh ra
- Có thể test API ngay trên trình duyệt

Flask: Phải viết tài liệu riêng (Word, Markdown)

- Hoặc dùng Flask-RESTX (cài thêm, phức tạp)
- Phải bảo trì 2 nơi: code + tài liệu

2.3. Kiểm tra kiểu dữ liệu và giá trị tự động

FastAPI:

```
1 @router.get("/by-club/{club}", summary="Lấy toàn bộ cầu thủ thuộc CLB")
2 async def players_of_club(
3     club: str = Path(..., description="Tên câu lạc bộ, ví dụ: 'Arsenal'"),
4     exact: bool = Query(True, description="True = khớp chính xác; False = LIKE search"),
5     db: aioredis.Connection = Depends(get_db),
6 ) -> List[Dict[str, Any]]:
7     """Lấy toàn bộ chỉ số của tất cả cầu thủ thuộc CLB"""
8     rows = await fetch_by_club(db, club=club, exact=exact)
9     if not rows:
10         raise HTTPException(status_code=404, detail=f"Không tìm thấy cầu thủ thuộc CLB '{club}'.")
11     return rows
```

Lợi ích:

- Khai báo kiểu: Editor tự động gợi ý, phát hiện lỗi trước khi chạy
- Tự động kiểm tra: exact phải là true/false, tự động chuyển đổi
- Thông báo lỗi tự động: {"detail": "value is not a valid boolean"}
- Editor biết cấu trúc dữ liệu trả về

2.4. Hệ thống tiêm phụ thuộc (Dependency Injection)

FastAPI:

```
1 @router.get("", summary="Tìm cầu thủ theo tên")
2 async def by_name(
3     name: str = Query(..., description="Tên cầu thủ, ví dụ: 'haaland'"),
4     db: aiosqlite.Connection = Depends(get_db),
5 ) -> List[Dict[str, Any]]:
```

Lợi ích:

- Tự động chèn kết nối database
- Tự động dọn dẹp (đóng kết nối)
- Dễ test (có thể giả lập phụ thuộc)
- Code sạch, không lặp lại

